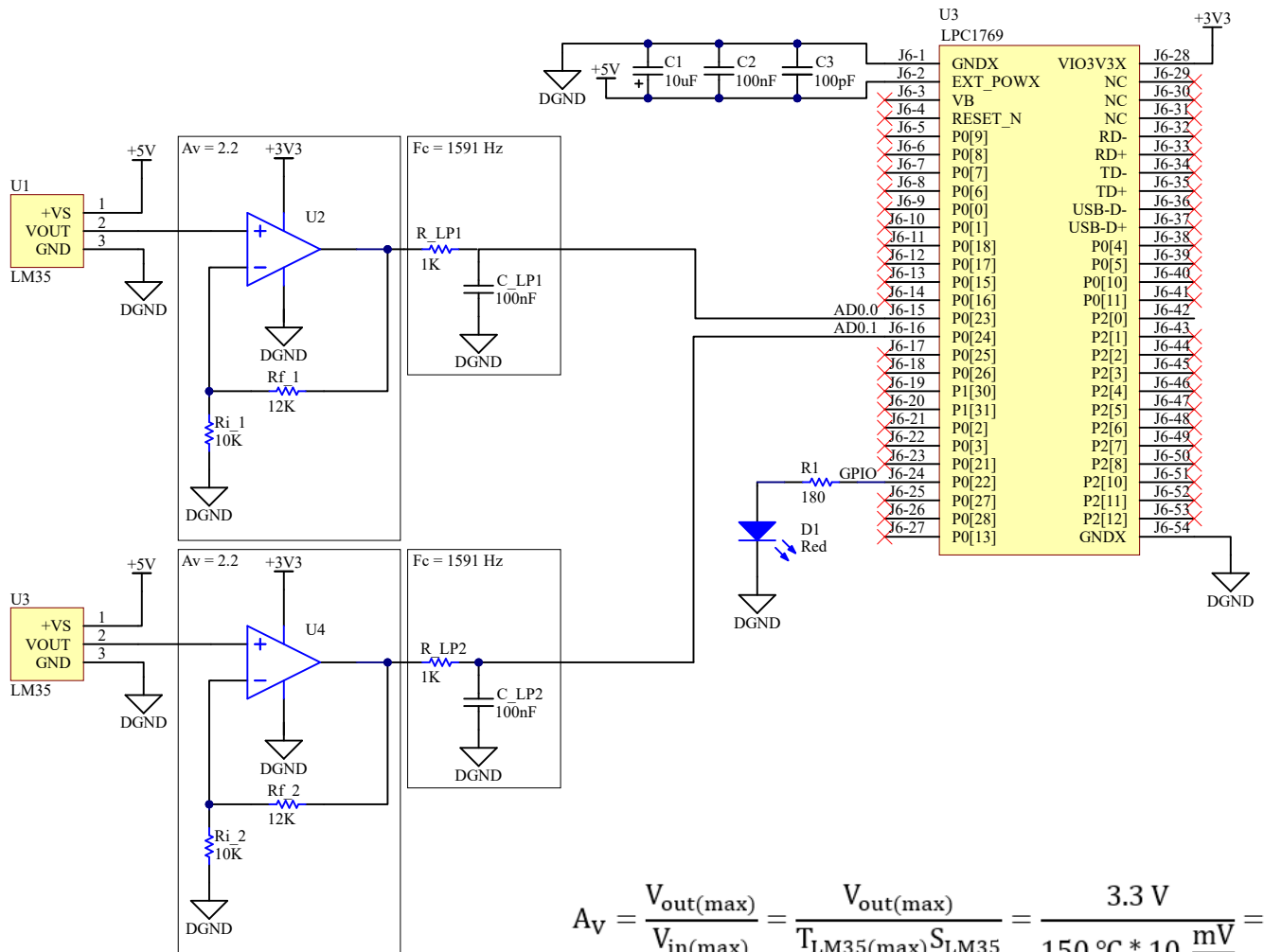


```

1  /*=====
2  * @file     Ejercicio_6_2_2.c
3  *
4  * @brief     Se requiere monitorizar la temperatura ambiente mediante dos
5  *            sensores analógicos LM35 conectados a los canales AD0.0 y AD0.1
6  *            del ADC.
7  *            Realizar el muestreo periódico de ambas señales y detectar el
8  *            sobrepaso de un umbral crítico de temperatura de 60 °C mediante
9  *            alarma técnica visual de prioridad alta, conformada por un LED rojo
10 *
11 *
12 * @details   - Las señales de los sensores LM35 se acondicionan mediante
13 *             amplificadores operacionales configurados para maximizar el
14 *             aprovechamiento del rango dinámico del ADC.
15 *             - El acondicionamiento adapta la tensión generada por los
16 *             sensores al rango de entrada del LPC1769 (0 V a 3.3 V),
17 *             incrementando la resolución efectiva de la medición y
18 *             mejorando la inmunidad al ruido durante la adquisición.
19 *             - Sensibilidad LM35: lineal de 10 mV/°C.
20 *             - Base de tiempo de adquisición: 1 muestra por segundo (1 Hz)
21 *             mediante el disparo por hardware de MAT0.1 del Timer 0.
22 *             - Al finalizar AD0.0, la ISR inicia por software la conversión
23 *             del canal AD0.1.
24 *             - Implementación en alto nivel mediante drivers (NXP LPC17xx).
25 *
26 * @author    Ing. E. Migliore
27 *
28 * @version   1.0
29  */

```



$$A_V = \frac{V_{out(max)}}{V_{in(max)}} = \frac{V_{out(max)}}{T_{LM35(max)} S_{LM35}} = \frac{3.3 \text{ V}}{150 \text{ }^{\circ}\text{C} * 10 \frac{\text{mV}}{^{\circ}\text{C}}} = 2.2$$

$$A_V = 1 + \frac{R_f}{R_i} \Rightarrow R_f = (A_V - 1)R_i = (2.2 - 1) * 10 \text{ k}\Omega = 12 \text{ k}\Omega$$

```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5  #include "lpc17xx_gpio.h"
6  #include "lpc17xx_pinsel.h"
7  #include "lpc17xx_timer.h"
8  #include "lpc17xx_adc.h"
9  #include <stdbool.h>
10
11 /*=====
12 * DEFINICIÓN DE CONSTANTES
13 *=====*/
14 /* Parámetros del ADC */
15 #define ADC_VREF_MV          3300U
16 #define ADC_MAX_COUNTS      4095U
17 #define ADC_SAMPLE_RATE_HZ  200000U
18
19 /* Parámetros de adquisición */
20 #define ADC_TRIGGER_RATE_HZ  1U
21 #define TMR0_PR_VALUE       1U
22 #define US_PER_SEC          1000000U
23 #define TMR0_MR_VALUE \
24     ((US_PER_SEC / (2U * ADC_TRIGGER_RATE_HZ * TMR0_PR_VALUE)) - 1U)
25
26 /* Parámetros de sensores LM35 */
27 #define LM35_SENSITIVITY_MV  10U
28
29 /* Canales ADC */
30 #define LM35_SENSOR_1_CHANNEL ADC_CHANNEL_0
31 #define LM35_SENSOR_2_CHANNEL ADC_CHANNEL_1
32
33 /* Pines de salida digital */
34 #define ALARM_HIGH_PORT      PINSEL_PORT_0
35 #define ALARM_HIGH_PIN      PINSEL_PIN_22
36 #define ALARM_HIGH_PIN_MASK (1U << 22)
37
38 /* Modalidades GPIO */
39 #define GPIO_INPUT           0U
40 #define GPIO_OUTPUT          1U
41
42 /* Umbrales de supervisión */
43 #define ALARM_TEMP_C         60U
44
45 /*=====
46 * DEFINICIÓN DE TIPOS
47 *=====*/
48 /**
49  * @brief Estado de la secuencia de adquisición de los canales ADC.
50  */
51 typedef enum
52 {
53     ADC_ACQ_SENSOR_1 = 0,
54     ADC_ACQ_SENSOR_2
55 } ADC_ACQ_STATE_Type;
56
57 /*=====
58 * DEFINICIÓN DE VARIABLES GLOBALES
59 *=====*/
60
61 volatile ADC_ACQ_STATE_Type adc_acq_state = ADC_ACQ_SENSOR_1;
62 volatile bool adc_flag_ready = false;
63
64 volatile uint16_t lm35_1_raw = 0;
65 volatile uint32_t lm35_1_mv = 0;
66 volatile uint32_t lm35_1_temp_c = 0;
67
68 volatile uint16_t lm35_2_raw = 0;
69 volatile uint32_t lm35_2_mv = 0;

```

```

70 volatile uint32_t lm35_2_temp_c = 0;
71
72 /*=====
73  * PROTOTIPOS DE FUNCIONES
74  *=====*/
75 void cfgGPIO(void);
76 void cfgTMR0(uint32_t period_us, uint32_t match_value, uint8_t mode);
77 void cfgADC(uint32_t rate);
78
79 static uint32_t adcRawToMillivolts(uint16_t adc_raw);
80 static uint32_t lm35MillivoltsToCelsius(uint32_t voltage_mv);
81 static void updateTemperatureAlarm(void);
82
83 /*=====
84  * FUNCIÓN PRINCIPAL
85  *=====*/
86 int main(void)
87 {
88     /*-----
89     * INICIALIZACIÓN DEL MCU
90     *-----*/
91     cfgGPIO();
92     cfgADC(ADC_SAMPLE_RATE_HZ);
93     cfgTMR0(TMR0_PR_VALUE, TMR0_MR_VALUE, TIM_EXTMATCH_TOGGLE);
94
95     while(1){
96         /*
97          * El procesamiento principal puede ejecutarse aquí.
98          * Las conversiones y la gestion de salidas se realizan por interrupción.
99          * La bandera indica que se proceso un nuevo par lm35_1_temp/lm35_2_temp.
100          */
101         if (adc_flag_ready == true) {
102             adc_flag_ready = false;
103             // ...
104         }
105     }
106 }
107
108 /*=====
109  * DEFINICIONES DE FUNCIONES
110  *=====*/
111 /*****
112  * @brief Configura las direcciones GPIO, modos de función y resistencias.
113  *
114  * @details - P0.22: Función GPIO como Salida Digital (Alarma de prioridad alta)
115  *           - P0.23: Función AD0.0 como Entrada Analógica (Sensor LM35 N° 1)
116  *           - P0.24: Función AD0.1 como Entrada Analógica (Sensor LM35 N° 2)
117  *           - P1.29: Función MAT0.1 como Salida (Match Timer 0).
118  *
119  * @param None
120  * @return None
121  *****/
122 void cfgGPIO(void)
123 {
124     PINSEL_CFG_Type cfgPinAlarmHigh;
125     PINSEL_CFG_Type cfgPinSensorTemp1;
126     PINSEL_CFG_Type cfgPinSensorTemp2;
127     PINSEL_CFG_Type cfgPinTriggerADC;
128
129     /* P0.22 como GPIO para alarma de prioridad alta */
130     cfgPinAlarmHigh.Portnum = PINSEL_PORT_0;
131     cfgPinAlarmHigh.Pinnum = PINSEL_PIN_22;
132     cfgPinAlarmHigh.Funcnum = PINSEL_FUNC_0;
133     cfgPinAlarmHigh.Pinmode = PINSEL_PINMODE_PULLDOWN;
134     cfgPinAlarmHigh.OpenDrain = PINSEL_PINMODE_NORMAL;
135
136     /* P0.23 como AD0.0 para el LM35 N° 1 */
137     cfgPinSensorTemp1.Portnum = PINSEL_PORT_0;
138     cfgPinSensorTemp1.Pinnum = PINSEL_PIN_23;

```

```

139     cfgPinSensorTemp1.Funcnum    = PINSEL_FUNC_1;
140     cfgPinSensorTemp1.Pinmode    = PINSEL_PINMODE_TRISTATE;
141     cfgPinSensorTemp1.OpenDrain  = PINSEL_PINMODE_NORMAL;
142
143     /* P0.24 como AD0.1 para el LM35 N° 2 */
144     cfgPinSensorTemp2.Portnum    = PINSEL_PORT_0;
145     cfgPinSensorTemp2.Pinnum     = PINSEL_PIN_24;
146     cfgPinSensorTemp2.Funcnum    = PINSEL_FUNC_1;
147     cfgPinSensorTemp2.Pinmode    = PINSEL_PINMODE_TRISTATE;
148     cfgPinSensorTemp2.OpenDrain  = PINSEL_PINMODE_NORMAL;
149
150     /* P1.29 como MAT0.1 */
151     cfgPinTriggerADC.Portnum     = PINSEL_PORT_1;
152     cfgPinTriggerADC.Pinnum      = PINSEL_PIN_29;
153     cfgPinTriggerADC.Funcnum     = PINSEL_FUNC_3;
154     cfgPinTriggerADC.Pinmode     = PINSEL_PINMODE_PULLUP;
155     cfgPinTriggerADC.OpenDrain   = PINSEL_PINMODE_NORMAL;
156
157     PINSEL_ConfigPin(&cfgPinAlarmHigh);
158     PINSEL_ConfigPin(&cfgPinSensorTemp1);
159     PINSEL_ConfigPin(&cfgPinSensorTemp2);
160     PINSEL_ConfigPin(&cfgPinTriggerADC);
161
162     GPIO_SetDir(ALARM_HIGH_PORT, ALARM_HIGH_PIN_MASK, GPIO_OUTPUT);
163     GPIO_ClearValue(ALARM_HIGH_PORT, ALARM_HIGH_PIN_MASK);
164
165     return;
166 }
167
168 /*****
169  * @brief   Configura e inicializa el periférico TIMER0 en modo Timer.
170  *
171  * @details Genera una señal periódica en MAT0.1 destinada al disparo
172  *           por hardware del ADC.
173  *
174  *           La frecuencia de la señal generada depende de la base de tiempo
175  *           configurada para el Prescale Counter (PC), del valor de
176  *           coincidencia MR1 y de la modalidad de salida seleccionada para
177  *           MAT0.1.
178  *
179  * @param   <period_us> Tiempo de desborde del Prescale Counter (PC),
180  *               expresado en microsegundos.
181  * @param   <match_value> Valor límite del registro de coincidencia MR1.
182  * @param   <mode> Modalidad de salida External Match Control (EMC1)
183  *               aplicada sobre MAT0.1.
184  *
185  * @return  None.
186  *****/
187 void cfgTMR0(uint32_t period_us, uint32_t match_value, uint8_t mode)
188 {
189     TIM_TIMERCFG_Type cfgTimerMode;
190     TIM_MATCHCFG_Type cfgTimerMatch;
191
192     cfgTimerMode.PrescaleOption = TIM_PRESCALE_USVAL;
193     cfgTimerMode.PrescaleValue  = period_us;
194
195     cfgTimerMatch.MatchChannel    = 1;
196     cfgTimerMatch.MatchValue     = match_value;
197     cfgTimerMatch.IntOnMatch     = DISABLE;
198     cfgTimerMatch.ResetOnMatch   = ENABLE;
199     cfgTimerMatch.StopOnMatch    = DISABLE;
200     cfgTimerMatch.ExtMatchOutputType = mode;
201
202     TIM_Init(LPC_TIM0, TIM_TIMER_MODE, &cfgTimerMode);
203     TIM_ConfigMatch(LPC_TIM0, &cfgTimerMatch);
204
205     NVIC_DisableIRQ(TIMER0_IRQn);
206
207     TIM_Cmd(LPC_TIM0, ENABLE);

```

```

208
209     return;
210 }
211
212 /*****
213  * @brief   Configura e inicializa el periférico ADC.
214  *
215  * @details Configura la conversión del canal AD0.0 disparada por flanco
216  *           descendente en el hardware MAT0.1 del Timer 0.
217  *           El canal AD0.1 permanece deshabilitado hasta que finaliza la
218  *           conversión de AD0.0. Su conversión se inicia por software desde
219  *           la ISR del ADC.
220  *
221  * @param   <rate> Tasa de conversión del ADC [S/s]
222  *
223  * @return  None.
224  *****/
225 void cfgADC(uint32_t rate)
226 {
227     ADC_Init(LPC_ADC, rate);
228     ADC_BurstCmd(LPC_ADC, DISABLE);
229     ADC_StartCmd(LPC_ADC, ADC_START_ON_MAT01);
230     ADC_EdgeStartConfig(LPC_ADC, ADC_START_ON_FALLING);
231     ADC_ChannelCmd(LPC_ADC, LM35_SENSOR_1_CHANNEL, ENABLE);
232     ADC_ChannelCmd(LPC_ADC, LM35_SENSOR_2_CHANNEL, DISABLE);
233     ADC_IntConfig(LPC_ADC, ADC_ADINTEN0, ENABLE);
234     ADC_IntConfig(LPC_ADC, ADC_ADINTEN1, ENABLE);
235
236     adc_acq_state = ADC_ACQ_SENSOR_1;
237
238     NVIC_EnableIRQ(ADC_IRQn);
239
240     return;
241 }
242
243 /*****
244  * @brief   Convierte el resultado digital del ADC a milivoltios.
245  *
246  * @param   <adc_raw> Resultado digital del ADC de 12 bits.
247  *
248  * @return  Tensión de entrada expresada en milivoltios.
249  *****/
250 static uint32_t adcRawToMillivolts(uint16_t adc_raw)
251 {
252     return (((uint32_t)adc_raw * ADC_VREF_MV) / ADC_MAX_COUNTS);
253 }
254
255 /*****
256  * @brief   Convierte la salida del LM35 de milivoltios a grados Celsius.
257  *
258  * @param   <voltage_mv> Tensión de salida del LM35 en milivoltios.
259  *
260  * @return  Temperatura calculada en grados Celsius enteros.
261  *****/
262 static uint32_t lm35MillivoltsToCelsius(uint32_t voltage_mv)
263 {
264     return (voltage_mv / LM35_SENSITIVITY_MV);
265 }
266
267 /*****
268  * @brief   Actualiza el estado de la alarma visual de temperatura.
269  *
270  * @details El LED se activa si al menos uno de los dos sensores registra
271  *           una temperatura superior al umbral crítico de 60 °C.
272  *
273  * @param   None.
274  * @return  None.
275  *****/
276 static void updateTemperatureAlarm(void)

```

```

277 {
278     if ((lm35_1_temp_c > ALARM_TEMP_C) ||
279         (lm35_2_temp_c > ALARM_TEMP_C))
280     {
281         GPIO_SetValue(ALARM_HIGH_PORT, ALARM_HIGH_PIN_MASK);
282     }
283     else
284     {
285         GPIO_ClearValue(ALARM_HIGH_PORT, ALARM_HIGH_PIN_MASK);
286     }
287 }
288
289 /*****
290  * @brief   Rutina de Servicio de Interrupción (ISR) para el vector ADC.
291  *
292  * @details Implementa una secuencia de adquisición de dos etapas:
293  *           1. Lee AD0.0 después del disparo de MAT0.1.
294  *           2. Selecciona AD0.1 e inicia su conversión por software.
295  *           3. Lee AD0.1 y procesa ambos sensores.
296  *           4. Restaura AD0.0 y el disparo mediante MAT0.1.
297  *
298  * @param   None.
299  * @return  None.
300  *****/
301 void ADC_IRQHandler(void)
302 {
303     if (adc_acq_state == ADC_ACQ_SENSOR_1){
304         if (ADC_ChannelGetStatus(LPC_ADC, LM35_SENSOR_1_CHANNEL, ADC_DATA_DONE) == SET){
305             // Lectura del LM35 N°1
306             lm35_1_raw = ADC_ChannelGetData(LPC_ADC, LM35_SENSOR_1_CHANNEL);
307
308             // Conmutación de canales: AD0.0 -> AD0.1
309             ADC_ChannelCmd(LPC_ADC, LM35_SENSOR_1_CHANNEL, DISABLE);
310             ADC_ChannelCmd(LPC_ADC, LM35_SENSOR_2_CHANNEL, ENABLE);
311             adc_acq_state = ADC_ACQ_SENSOR_2;
312
313             // Inicio conversión por software del LM35 N°2 (AD0.1)
314             ADC_StartCmd(LPC_ADC, ADC_START_NOW);
315         }
316     }
317     else{
318         if (ADC_ChannelGetStatus(LPC_ADC, LM35_SENSOR_2_CHANNEL, ADC_DATA_DONE) == SET){
319             // Lectura del LM35 N°2
320             lm35_2_raw = ADC_ChannelGetData(LPC_ADC, LM35_SENSOR_2_CHANNEL);
321
322             // Conversión de Cuentas AD a Tensión de Entrada [mV]
323             lm35_1_mv = adcRawToMillivolts(lm35_1_raw);
324             lm35_2_mv = adcRawToMillivolts(lm35_2_raw);
325
326             // Conversión de Tensión de Entrada a Temperatura [°C]
327             lm35_1_temp_c = lm35MillivoltsToCelsius(lm35_1_mv);
328             lm35_2_temp_c = lm35MillivoltsToCelsius(lm35_2_mv);
329
330             // Procesamiento del Dato Acondicionado en Unidad Física [°C]
331             updateTemperatureAlarm();
332
333             adc_flag_ready = true;
334
335             // Conmutación de canales: AD0.1 -> AD0.0
336             ADC_ChannelCmd(LPC_ADC, LM35_SENSOR_1_CHANNEL, ENABLE);
337             ADC_ChannelCmd(LPC_ADC, LM35_SENSOR_2_CHANNEL, DISABLE);
338             adc_acq_state = ADC_ACQ_SENSOR_1;
339
340             ADC_StartCmd(LPC_ADC, ADC_START_ON_MAT01);
341             ADC_EdgeStartConfig(LPC_ADC, ADC_START_ON_FALLING);
342         }
343     }
344 }

```